

Object oriented and object relational databases

OODBMS

The object-oriented database system is an extension of an [object-oriented programming language](#) that includes DBMS functions such as persistent objects, integrity constraints, failure recovery, transaction management, and query processing. These systems feature object description language (ODL) for database structure creation and object query language (OQL) for database querying.

Some examples of OODBMS are ObjectStore, Objectivity/DB, GemStone, db4o, Giga Base, and Zope object database.

Advantages of OODBMS

Complex Data Representation: This is because, through OODBMS, hard dependencies can be well modeled to capture flexibility such as inheritance, encapsulation, and polymorphism.

Direct Mapping: Here, objects of OODBMS match with the objects in programming languages, and so there is less difference between them, which is known as impedance.

Reusability: Lectures such as inheritance make it possible to use existing code thereby shortening development time.

Disadvantages of OODBMS

Complexity: The key working principle in an OODBMS is that the OODBMS is an implementation level of OOD, therefore, is more complex than traditional relational databases.

Limited Query Support: OODBMS does not support queries such as SQL meaning there is little flexibility when it comes to querying.

Lack of Standardization: OODBMS do not contain such standards as is the case with relational databases hence making the interfaces across different platforms to be inconsistent.

ORDBMS

An object-relational database system is a [relational database system](#) that has been extended to incorporate object-oriented characteristics. Database schemas and the query language natively support objects, classes, and inheritance. Furthermore, it permits data model expansion with new data types and procedures, exactly like pure relational systems. Oracle, DB2, Informix, [PostgreSQL](#) (UC Berkeley research project), etc. are some of the ORDBMSs.

Advantages of ORDBMS

- SQL Support:** While ORDBMS supports the structures of the SQL query language, they are quite flexible when tested against query language make it easier to handle complex queries unlike OODBMS.
- Scalability:** It is easier and can accommodate large data sets as compared to OODBMS due to the fact that it retains the structure of relational databases.
- Interoperability:** ORDBMS appears to offer better compatibility with the conventional RDMS thus making it easy to migrate from the conventional normal database to more OOD respectively.

Disadvantages of ORDBMS

- Complexity in Implementation:** Although many object-oriented notions are supported by ORDBMS their representation using [SQL](#) can be problematic.
- Overhead:** The combination in a hybrid approach causes a certain overhead which does have its influence in cases where only object-oriented or tabular representations are needed.

Difference Between OODBMS and ORDBMS

OODBMS	ORDBMS
It stands for Object Oriented Database Management System.	It stands for Object Relational Database Management System.
Object-oriented databases, like Object Oriented Programming, represent data in the form of objects and classes.	An object-relational database is one that is based on both the relational and object-oriented database models.
OODBMSs support ODL/OQL.	ORDBMS adds object-oriented functionalities to SQL.
Every object-oriented system has a different set of constraints that it can accommodate.	Keys, entity integrity, and referential integrity are constraints of an object-oriented database.
The efficiency of query processing is low.	Processing of queries is quite effective.

Conclusion

OODBMS and ORDBMS both are used for different requirements and applications are well suited for them. OODBMS is used for the system which needs objects to be mapped directly and when the structure of the data is complex.

ORDBMS is suited for benefits of relational databases but with little of OOP. Therefore, the choice between these two systems depends on certain aspects of the application, for example, need for query capabilities or object orientation.

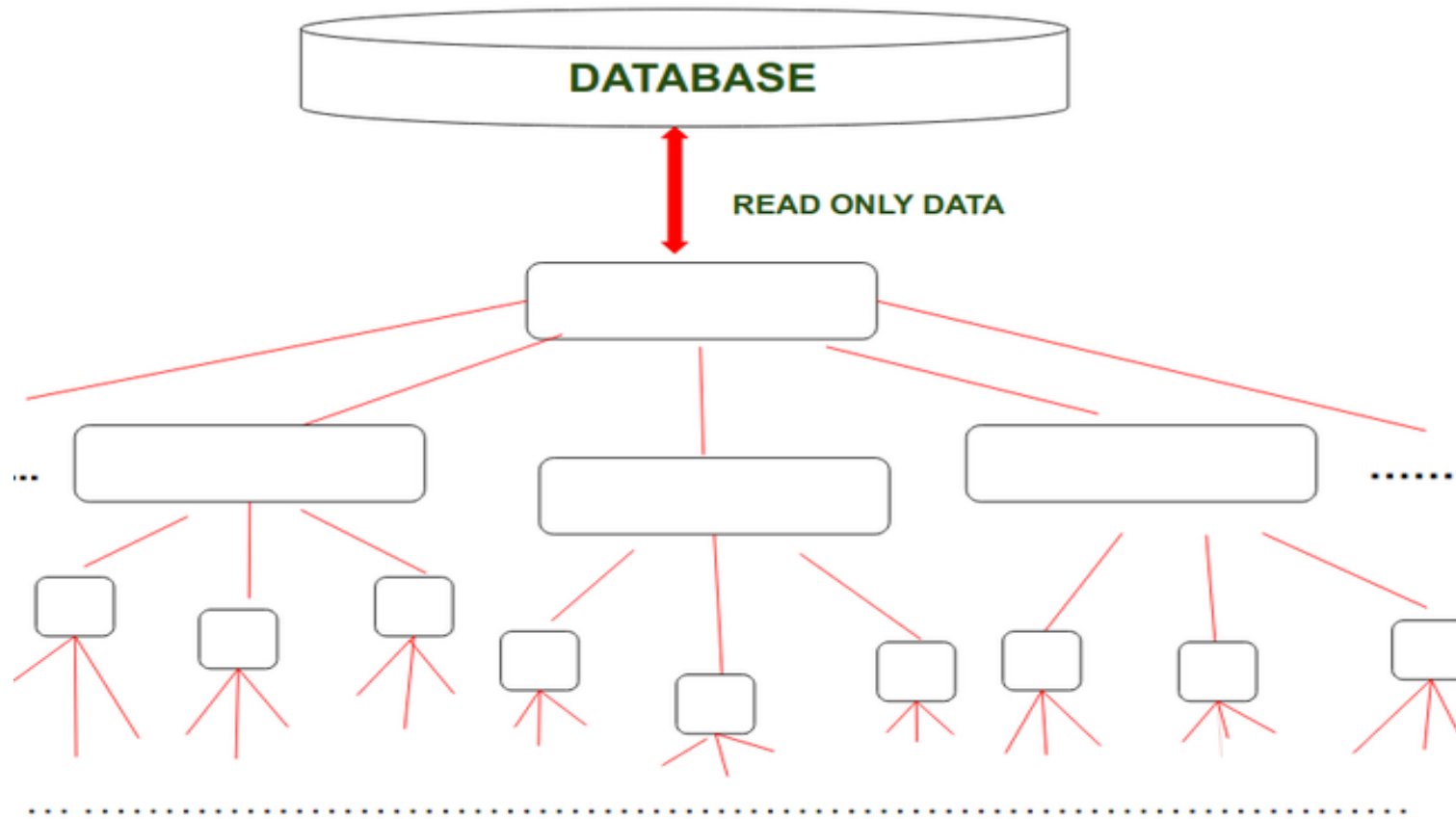
Logical databases

A Logical Database is a special type of ABAP (Advance Business Application and Programming) that is used to retrieve data from various tables and the data is interrelated to each other.

A logical database provides a read-only view of Data.

A Logical database uses only a hierarchical structure of tables i.e. Data is organized in a Tree-like Structure and the data is stored as records that are connected to each other through edges (Links).

Logical Database contains Open SQL statements which are used to read data from the database



Structure of a Logical Database

Some more points to be noted on Logical Databases

- We can select only that type of Data that we need.
- Data Authentication is done in order to maintain security.
- Logical Database uses hierarchical Structure due to this data integrity is maintained.

Goal of Logical Database:

The goal of Logical Database is to create well-structured tables that reflect the need of the user.

The tables of the Logical database store data in a non-redundant manner and foreign keys are used in tables so that relationships among tables and entities will be supported.

Tasks of Logical Database:

- ❑ With the help of the Logical database, we can read the same data from multiple programs.
- ❑ A logical database defines the same user interface for multiple programs.
- ❑ Logical Database ensures the Authorization checks for the centralized sensitive database.
- ❑ With the help of a Logical Database, performance is improved.
For example in Logical Database we will use joins instead of multiple SELECT statements, which will improve response time and this will increase the Performance of Logical Database.

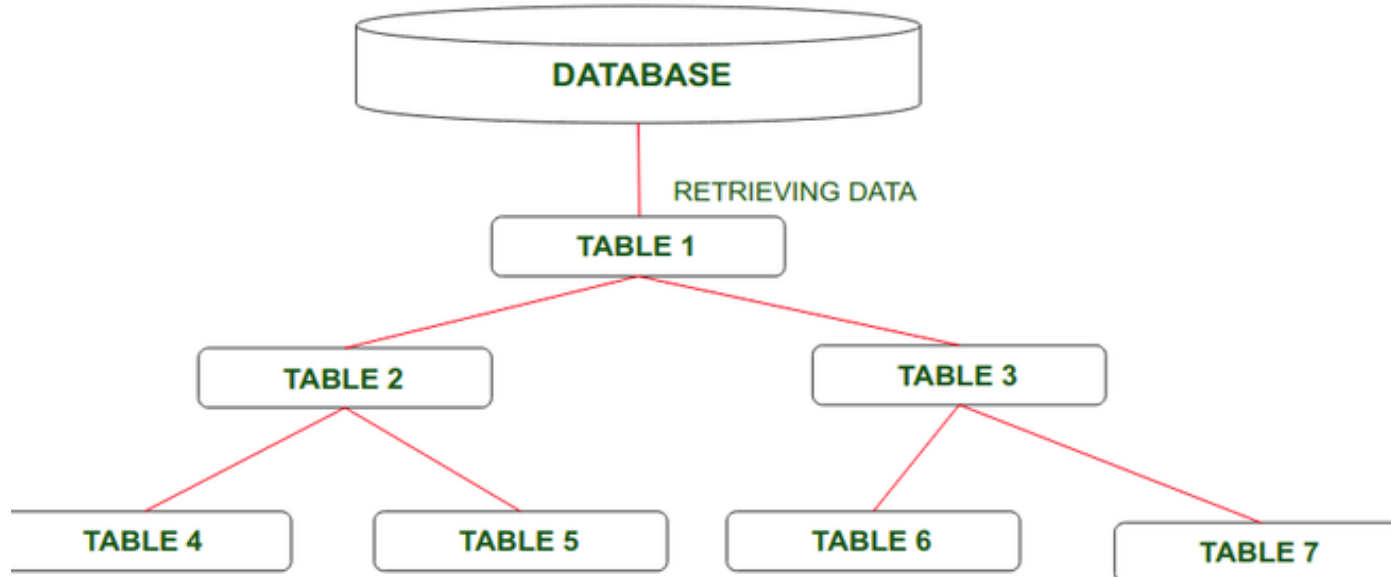
Data View Of Logical Database:

Logical Database provides a particular view of Logical Database tables.

A logical database is appropriately used when the structure of the Database is Large. It is convenient to use flow i.e

- SELECT
- READ
- PROCESS
- DISPLAY

In order to work with databases efficiently. The data of the Logical Database is hierarchical in nature. The tables are linked to each other in a Foreign Key relationship.



Advantages Of Logical Database:

- In a Logical database, we can select meaningful data from a large amount of data.
- Logical Database consists of Central Authorization which checks for Database Accesses is Authenticated or not.
- In this Coding, the part is less required to retrieve data from the database as compared to Other Databases.
- Access performance of reading data from the hierarchical structure of the Database is good.
- Easy to understand user interfaces.
- Logical Database firstly check functions which further check that user input is complete, correct, and plausible.

Disadvantage Of Logical Database:

- Logical Database takes more time when the required data is at the last because if that table which is required at the lowest level then firstly all upper-level tables should be read which takes more time and this slows down the performance.

Web Databases

Web databases are organized collections of data that are hosted on a server and accessible through the internet or a local network.

They act as the "backbone" of dynamic web applications, allowing sites to store user profiles, process transactions, and serve real-time content rather than just displaying static, unchanging information.

Core Functionality

When you interact with a website (e.g., clicking a "Submit" button), a three-layer process typically occurs:

1.Frontend (User Request): The user triggers an action in their web browser.

2.Application Tier (Backend): The web server receives the request and processes it using business logic to formulate a database query.

1.Database Tier: The database executes the query, retrieves or updates the data, and sends the result back through the application tier to be displayed to the user

Popular Web Database Systems

Modern development relies on several types of databases, each suited for different workloads:

- **Relational (SQL):** Best for structured data and complex relationships.

- [MySQL](#): Widely used, open-source, and known for stability.
- [PostgreSQL](#): Advanced relational database famous for its robustness and complex query support.
- **SQLite**: A lightweight, serverless database often used for smaller applications.

- **NoSQL / Cloud-Based:** Best for high scalability and flexible data structures.

- [MongoDB](#): A document-oriented platform designed for modern, high-speed data handling.
- **Firebase (Firestore)**: A cloud-based real-time database often used for mobile and web apps.
- **Redis**: An in-memory key-value store used primarily for caching and session management to speed up performance

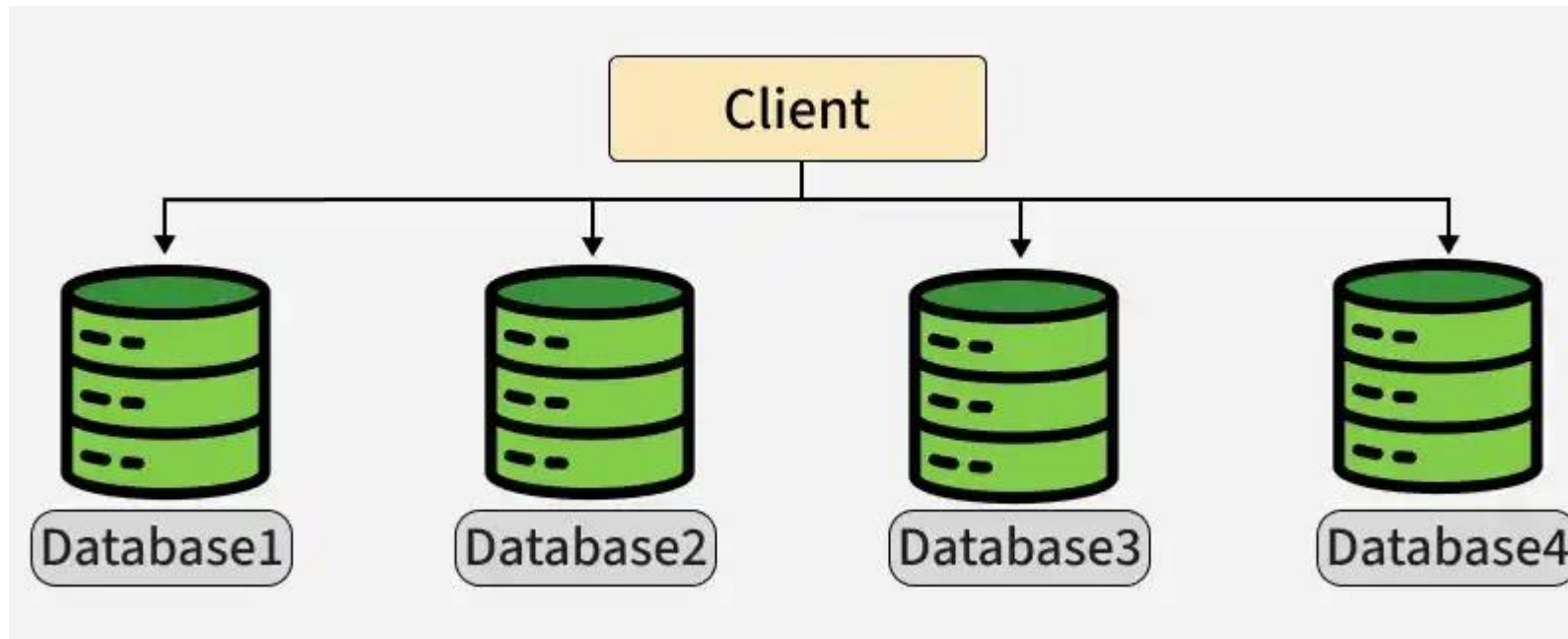
Key Benefits

- Scalability:** Cloud-hosted databases can grow as user traffic increases.
- Data Integrity:** Features like ACID compliance ensure data remains accurate and consistent even during complex transactions.
- Security:** Most systems include built-in encryption, authentication, and access control to protect sensitive user data.
- Accessibility:** Data is available 24/7 from any device with an internet connection.

Distributed databases

A Distributed Database System (DDBS) is a collection of multiple databases spread across different physical locations, connected via a network.

- Data is stored across multiple sites but appears as a single database to users.
- Enables local access to data, improving response time and performance.
- Supports parallel processing, allowing multiple operations at the same time.



1. Homogeneous Database:

In a homogeneous database, all different sites store database identically. The operating system, database management system, and the data structures used all are the same at all sites.

Features:

- Unified query language and interface.
- Low integration complexity.
- Efficient synchronization.

Example: A bank with branches in different cities uses Oracle DB at every location. All databases have the same structure and are synchronized regularly.

2. Heterogeneous Database

In a heterogeneous distributed database, different sites may use different DBMSs, schemas, or data models, making query processing and transactions difficult. Some sites may not even be aware of others, so translation mechanisms are needed for communication.

Features:

- Supports interoperability between diverse systems.
- Complex query optimization and transaction management.
- Useful in mergers or collaborations between organizations.

Example: A logistics company uses MySQL for inventory, MongoDB for vehicle tracking, and PostgreSQL for billing. Integration middleware allows unified querying across these platforms.

3. Client-Server Distributed Database System

The server stores and manages the database, while clients send queries over the network. It offers centralized control with distributed access, making it ideal for enterprise systems and web applications. Clients can be lightweight while the server handles heavy processing.

Example: Web application interacting with a central PostgreSQL server.

Features:

- Simplifies resource management.
- Central servers can be optimized for performance.
- Easily scalable with more clients.

Example: An e-commerce website where the frontend (client) is hosted separately and interacts with a central PostgreSQL server to manage orders, users, and inventory.

4. Peer-to-Peer Distributed Database System

All nodes are equal, with no fixed client or server roles. Each node can store data and also process queries, leading to decentralized control. It supports fault tolerance and high availability.

Example: Blockchain networks like Ethereum, where each node maintains a part of the distributed ledger.

Features:

- No single point of failure.
- Useful in decentralized and distributed apps.
- High availability and data redundancy.

Example: Blockchain-based databases like Ethereum or BitTorrent-based systems, where each peer maintains part of the ledger and participates equally in transactions.

5. Cloud-Based Distributed Database System

These systems are deployed on cloud platforms and span multiple geographic regions for scalability and reliability. They abstract infrastructure details and are offered as **DBaaS**, making them ideal for dynamic workloads. **Example: Google Cloud Spanner and Amazon DynamoDB used for global applications.**

Features:

- Automatic scaling and replication.
- Pay-as-you-use pricing.
- Global availability and disaster recovery.

Example:

- Google Cloud Spanner**: Global-scale relational database.
- Amazon DynamoDB**: Key-value and document database with high performance.
- Azure Cosmos DB**: Multi-model, globally distributed DBMS.



Fragmentation



Concurrency
Control

Key Concepts in DDBMS



Replication



Semantic
Heterogeneity

1. Replication

Same data is stored at multiple sites for better availability and faster access. Full replication stores the entire database at every site but increases update and consistency overhead.

2. Fragmentation

Database tables are split and stored at different sites.

- Horizontal:** split by rows
- Vertical:** split by columns

Helps reduce redundancy and improves efficiency.

3. Concurrency Control

Ensures correct data when many transactions run together. Prevents issues like lost updates using methods like locking and timestamps.

4. Semantic Heterogeneity

Occurs when databases use different meanings, formats, or units for similar data. Needs resolution for proper integration.

Functions of Distributed Database:

- Corporate management systems
- Multimedia applications
- Military and hotel systems
- Manufacturing control systems

Advantages

Fast data processing
High reliability and availability
Easy system expansion
Better data sharing

Disadvantages

Complex to manage
Security challenges
Deadlock and consistency issues
Requires standardization

Data warehousing and Data mining

Data Warehousing:

Stores and organizes historical data from multiple sources using ETL[1] and OLAP[2]. Features: subject-oriented, integrated, time-variant, and non-volatile.

Data Mining:

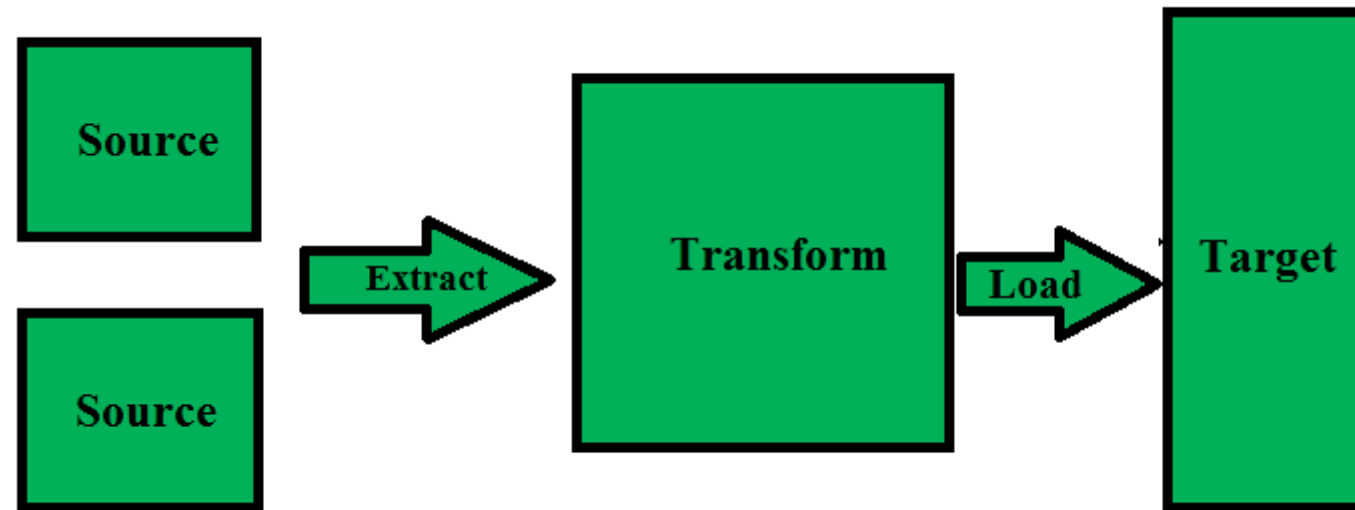
Analyzes large data to discover patterns and insights using classification, clustering, association rules, and regression.

Relationship:

Data warehouse provides organized data, and data mining extracts useful information from it.

[1] ETL (Extract, Transform, Load) is the process of gathering raw data from multiple sources, cleaning it, and loading it into a data warehouse for analysis.

[2] OLAP (Online Analytical Processing) is a technology used to analyze this prepared data, allowing fast, complex, multidimensional queries for business intelligence.



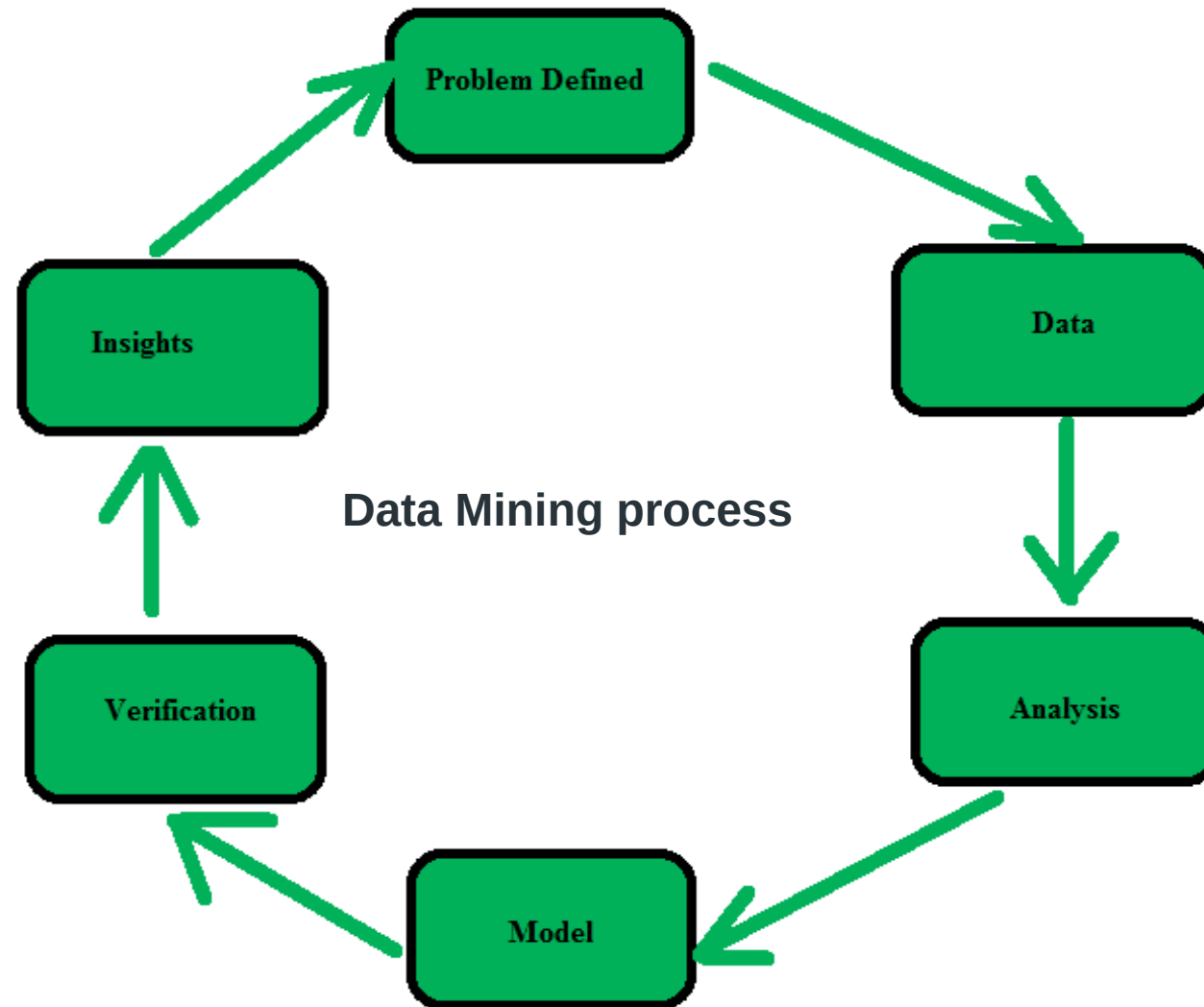
Data Warehousing process

Advantages of Data Warehousing

- Makes data easy to access and understand
- Supports continuous updates
- Stores large historical data for trend analysis and prediction

Disadvantages of Data Warehousing

- Risk of storing irrelevant or lost data
- Data cleaning and transformation from multiple sources is difficult



Advantages of Data Mining

- Helps analyze and sort data
- Detects faults and risks early
- Cost-effective and efficient
- Provides useful business insights

Disadvantages of Data Mining

- May not always be accurate
- Risk of data breaches
- Requires high training and implementation cost

Data Warehousing Vs Datamining

Basis	Data Warehousing	Data Mining
Definition	Stores organized data for analysis	Analyzes data to find patterns
Process	Data is stored periodically	Data is analyzed regularly
Purpose	Easier reporting and storage	Discover insights and predictions
Managed By	Engineers	Analysts/business users
Function	Integrated, historical data storage	Uses AI, statistics, ML techniques
Use	Improves reporting	Finds trends and customer patterns